

Q-Kernel: Contextuality Kernel Extraction for Weyl/Pauli Measurement Programs

Manuel Flores Gordillo

Draft v0.1 — internal working note

Abstract

We describe a small compiler-analysis primitive for Weyl/Pauli measurement programs: extraction of a minimum contextuality kernel. For even local dimension d , a Weyl context family has a state-independent all-versus-nothing obstruction precisely when some parity cycle of contexts carries an odd commutator-carry value. This reduces detection to binary linear algebra over the context–observable incidence matrix and kernel extraction to a minimum-weight odd-carry cycle problem. The resulting certificate is not proposed as an additive magic monotone or a T-count lower bound. Rather, it is a compact, exactly checkable obstruction certificate: a minimal subfamily of contexts that cannot admit a global noncontextual value assignment. We present the intended software architecture, the Peres–Mermin test case, and the next bridge questions needed before resource-estimation claims can be made.

1 Motivation

Stabilizer and Clifford-like fragments are algorithmically special: they admit efficient classical simulation in important regimes [1]. Non-Clifford resources and magic-state constructions are linked to contextuality in several settings [2, 3]. However, a direct scalar “amount of contextuality” is not always the right engineering object.

The goal here is narrower. Given a Weyl/Pauli measurement program, we want to extract the smallest obstruction-carrying subprogram:

$$\text{large measurement family} \quad \longmapsto \quad \text{minimal odd-carry contextual kernel.}$$

This is certificate compression, not a claim of universal quantum speedup.

2 Context families and carry bits

Fix an even local dimension d . Weyl observables are represented, up to phase, by labels

$$v \in \mathbb{Z}_d^{2m}.$$

We use the integer lift of the symplectic form

$$\langle v, w \rangle = \sum_k (v_{2k+1}w_{2k} - v_{2k}w_{2k+1}).$$

Two labels commute when $\langle v, w \rangle \equiv 0 \pmod{d}$.

A context $C = \{c_1, \dots, c_\ell\}$ is a finite pairwise-commuting set whose labels sum to zero modulo d . To each context we associate the commutator-carry bit

$$b(C) = \sum_{i < j} \frac{\langle c_i, c_j \rangle}{d} \pmod{2}.$$

The division is performed on the integer lift. Reducing $\langle c_i, c_j \rangle$ modulo d before division destroys the carry.

Let A be the context–observable incidence matrix over $\mathbb{F}_2$. Rows are contexts, columns are observables, and $A_{C,o} = 1$ when observable o appears in context C . A parity cycle is a vector $\lambda \in \mathbb{F}_2^{|F|}$ satisfying

$$A^T \lambda = 0.$$

Equivalently, every observable appears an even number of times across the selected contexts.

3 Odd-carry criterion

The implemented criterion is:

$$F \text{ is contextual} \iff \exists \lambda : A^T \lambda = 0, \quad b^T \lambda = 1.$$

In the companion mathematical note, this is identified with an intrinsic self-pairing:

$$b^T \lambda = Q(\lambda) = \sum_{a < b} \frac{\langle v_a, v_b \rangle}{d} \pmod{2},$$

where the sum runs over the observable multiset selected by the cycle. Thus the software can verify contextuality either as an incidence/carry computation or as an odd- Q certificate.

4 Kernelization problem

Detection asks whether any odd-carry cycle exists. Kernelization asks for the smallest one:

$$\min_{\lambda} \|\lambda\|_0 \quad \text{subject to} \quad A^T \lambda = 0, \quad b^T \lambda = 1.$$

The output is a contextuality kernel:

- the selected context indices;
- the selected observables;
- the odd Q -value;
- a compression ratio relative to the original family;
- a machine-checkable certificate.

This optimization has the shape of a minimum-weight coset/codeword problem over $\mathbb{F}_2$. Brute-force cycle enumeration is enough for small demonstrations, but larger compiler instances require branch-and-bound, MILP/CP-SAT, or sparse-cycle heuristics.

5 Peres–Mermin test case

For the two-qubit Peres–Mermin square, the six contexts are

$$\begin{array}{ccc} ZI & IZ & ZZ \\ IX & XI & XX \\ ZX & XZ & YY \end{array}$$

with the three row contexts and three column contexts. The carry vector is

$$b = (0, 0, 0, 0, 0, 1),$$

and the unique full parity cycle selects all six contexts:

$$\lambda = (1, 1, 1, 1, 1, 1).$$

Thus

$$b^T \lambda = 1,$$

which recovers the usual Peres–Mermin obstruction as an odd-carry kernel.

6 Schedule extraction caveat

A raw list of Pauli strings is not a measurement program. Contextuality depends on which observables are actually co-measured or jointly constrained. Therefore the first software frontend accepts grouped schedules:

$$\text{layers} = [[ZI, IZ, ZZ], [IX, XI, XX], \dots].$$

The extractor does not infer contexts across layers. This design prevents a false-positive failure mode where one generates all compatible triples from a mere observable set and thereby analyzes a measurement family that was never present in the circuit.

7 What is not claimed

This note deliberately avoids the following claims:

1. The kernel size is not yet a theorem-level lower bound on T -count, T -depth, magic-state injections, stabilizer rank, or distillation cost.
2. The obstruction is not an additive “contextual fuel gauge.” The companion note reads the odd- Q obstruction as a support/indicator of an anomaly-like structure, not as an additive anomaly charge.
3. The tool is not a general contextuality solver. It targets Weyl/Pauli context families in the regime covered by the odd-carry criterion.

8 Benchmark plan

The first empirical layer is intentionally modest. We do not attempt to infer quantum speedup or magic-state cost. Instead, we measure software properties of the kernel extractor on synthetic structured families: Peres–Mermin cores plus irrelevant disconnected checks, and connected zero-carry ladders. The diagnostics are runtime, number of connected components, kernel size, and independent certificate verification time.

These experiments are meant to answer a narrower question: do decomposition and solver choice preserve the contextual kernel while controlling the search problem? This is the correct prerequisite before connecting the invariant to resource-estimation metrics.

9 SAT/SMT direction

The minimum odd-carry kernel problem is naturally a Boolean optimization problem. Let $\lambda_i \in \{0, 1\}$ select contexts. Then the constraints are parity equations

$$\bigoplus_i A_{ij} \lambda_i = 0$$

for every observable j , together with the odd-carry constraint

$$\bigoplus_i b_i \lambda_i = 1.$$

The objective is to minimize $\sum_i \lambda_i$. This admits SAT, MaxSAT, SMT, MILP, or CP-SAT encodings. The implementation therefore includes a DIMACS CNF export using Tseitin encodings of parity constraints, so external SAT solvers can be used without becoming mandatory package dependencies. It also exports WCNF/MaxSAT instances in which the parity constraints are hard clauses and unit soft clauses penalize selected contexts, directly encoding the minimum-kernel objective. The solver-output import path then maps external assignments back to context-selection vectors and verifies them independently before producing a certificate. Such solvers should be treated as candidate generators: the returned λ is accepted only after the independent verifier checks closure and odd Q .

10 Bridge questions

The next research questions are:

1. Does minimum odd- Q kernel size correlate with standard compiler resources such as T -count, T -depth, magic injections, stabilizer rank, or distillation overhead?
2. Can restricted bridge theorems be proved for specific measurement-based or injection-based compilation models?
3. Can the $d \rightarrow 2d$ generativity phenomenon be turned into either a compiler warning or a useful construction?
4. What solver backend scales best for sparse, structured incidence matrices arising from real compiler outputs?

11 Component decomposition

A first scaling improvement comes from the context–observable bipartite graph. Its nodes are context nodes and observable nodes, with an edge when an observable appears in a context. If this graph decomposes into connected components, then every parity cycle decomposes as a sum of component cycles. Consequently, any odd-carry global cycle has odd carry in at least one component, and a minimum odd-carry certificate lives inside one connected component.

Thus the kernelizer can solve each connected component independently and keep the smallest odd- Q certificate. This is a simple but important distinction between a foundations example and a compiler-oriented implementation: unrelated measurement blocks should not enlarge the hard search problem.

12 Solver backends

The implementation separates certificate search from certificate verification. The current backends are deliberately simple.

The *span* backend enumerates the span of a cycle-space basis. It is exact and fast when the cycle dimension is small, but exponential in that dimension. The *bounded-weight* backend searches context subsets by increasing Hamming weight and is exact only up to a chosen weight/check budget. The current branch-and-bound baseline encodes each context as an incidence row concatenated with its carry bit and prunes a partial search whenever the remaining

suffix span cannot reach the residual target. This is still exponential in hard cases, but provides a dependency-free exact baseline.

The intended long-run architecture is solver-agnostic: branch-and-bound, MILP/CP-SAT, sparse-cycle heuristics, or meet-in-the-middle methods can propose a candidate λ , while the verifier independently checks $A^T\lambda = 0$ and $Q(\lambda) = 1$. Thus even heuristic solvers can produce exact certificates.

13 Observable identity and canonicalization

A practical compiler pass must distinguish algebraic equality from protocol identity. Two labels may represent the same Weyl vector while corresponding to different measurement events in a schedule. If the purpose is algebraic compression, such observables may be canonicalized by Weyl vector. If the purpose is protocol-faithful analysis, they should remain distinct unless the compiler frontend asserts that they are the same observable.

The implementation therefore supports explicit canonicalization modes. The default is no canonicalization. The optional by-vector mode merges equal Weyl labels when their identity scope is observable-level. Event-scoped labels remain distinct unless an explicit force mode is selected. The implementation also refuses to merge two observables inside the same context, since that would silently alter multiplicity semantics.

14 Certificates

The software separates search from trust by exporting standalone certificates. A certificate records a canonical hash of the input program, the selected vector λ , the selected context indices, the carry vector, the claimed Q -value, and metadata specifying the software version, coordinate convention and carry rule, and criterion identifier. Verification recomputes the program hash, checks that the selected contexts define the claimed λ , verifies $A^T\lambda = 0$, and recomputes $Q(\lambda) = 1$.

This design is important for future solver backends. A SAT, SMT, MILP, or heuristic solver may propose a candidate. The candidate is accepted only if the independent certificate verifier succeeds.

15 Adapter contract

The first compiler-facing adapter is deliberately dependency-free. It accepts a row-oriented Pauli table with explicit context identifiers and Pauli strings. This table is a stable boundary between external SDKs and the Q-Kernel core. A Qiskit, Stim, or compiler-specific frontend should translate its internal measurement representation into this table before invoking the odd- Q analyzer.

This avoids importing a heavy quantum SDK before the semantics are settled. More importantly, it forces each frontend to specify identity scope: repeated equal Pauli labels may represent either the same algebraic observable or distinct measurement events.

16 Software roadmap

The planned package structure is:

<code>ir.py</code>	program dataclasses,
<code>symplectic.py</code>	integer symplectic arithmetic,
<code>carry.py</code>	commutator-carry bits,
<code>incidence.py</code>	GF(2) incidence matrices,
<code>analyzer.py</code>	odd- Q detection,
<code>optimizer.py</code>	minimum kernel extraction,
<code>pauli.py</code>	Pauli-string frontend,
<code>pauli_schedule.py</code>	grouped schedule extraction.

References

- [1] S. Aaronson and D. Gottesman, *Improved simulation of stabilizer circuits*, Phys. Rev. A **70**, 052328 (2004).
- [2] M. Howard, J. Wallman, V. Veitch, and J. Emerson, *Contextuality supplies the magic for quantum computation*, Nature **510**, 351 (2014).
- [3] R. Raussendorf, C. Okay, M. Zurel, and H. Feldmann, *The role of cohomology in quantum computation with magic states*, Quantum **7**, 979 (2023).